Learning Rules from Examples: Reasoning and Evaluation in ARC-AGI-2

Nihal Sirivolu

sirivolun@gmail.com

Abstract. ARC-AGI-2 asks a system to infer an unfamiliar transformation from a small set of examples and construct an exact output grid. Its simple interface hides a difficult distinction: matching demonstrations does not establish that the intended rule has been learned. This paper synthesizes primary research on program induction, direct neural prediction, test-time training, and recursive refinement. It separates the benchmark's task format from its scoring protocol and explains why candidate generation, verification, and final selection must be evaluated separately. An analytical treatment of demonstration consistency and two-attempt scoring shows how apparently strong intermediate behavior can fail to produce correct predictions. Published competition results provide a bounded empirical reference, rather than a cross-benchmark ranking. The resulting evaluation framework emphasizes task-level separation, declared compute budgets, and traceable failure analysis. This is a literature-based research synthesis; no new solver, training run, or benchmark result is reported.

Keywords: Abstract reasoning · ARC-AGI-2 · Program synthesis · Test-time adaptation · Evaluation.

1 The Research Question

Learning a new rule differs from recognizing a familiar example. Chollet's original account of ARC frames intelligence in terms of acquiring skills efficiently, with prior knowledge and experience made explicit, rather than equating intelligence with a single task score [1]. ARC-AGI-2 continues this research direction through novel grid transformations and more demanding task construction [2].

The central question here is narrower than whether a system is generally intelligent: what evidence supports the claim that it inferred a rule from the examples it was given? We examine the published mechanisms, then derive practical consequences for evaluation. The discussion distinguishes three kinds of statements: documented benchmark requirements, results reported by other researchers, and our analytical interpretation. It is a focused synthesis, not an exhaustive systematic review. Sources were checked on 1 November 2026; historical results retain their original evaluation setting.

We examine representation, candidate construction, checking, and selection separately. A score identifies success; locating the failure stage makes that score useful for improving a system.

2 What the Task and Score Actually Measure

An ARC task supplies demonstration input–output pairs and new test inputs. The system must infer both the output dimensions and cell values. The official repository represents grids as rectangular arrays of integers from 0 to 9, with dimensions ranging from 1×1 to 30×30. It lists 1,000 public training tasks and 120 public evaluation tasks [3]. These are repository counts, not a newly measured audit of a downloaded competition release.

Two levels of data separation matter. A held-out task still includes its own demonstrations: using them is the problem definition. Its hidden test outputs are different; using them to tune the solver would invalidate an unseen-task evaluation. Table 1 keeps these roles separate.

Table 1. Information roles in ARC-AGI-2. Counts follow the official public repository [3]; task-local examples are not additional dataset splits.

Collection	Published size	Appropriate role
Public training	1,000 tasks	Learn priors and develop the system.
Public evaluation	120 tasks	Evaluate a frozen method; disclose prior exposure.
Task demonstrations	Task-dependent	Infer the current transformation.
Hidden test outputs	Unavailable to solver	Score predictions after they are fixed.

For the 2026 competition, each test input receives exactly two candidate outputs. A prediction succeeds only if a candidate matches the entire reference grid, including its shape. The score averages success over test outputs, not pixels [4]. Let task t have m_t test inputs, reference y_{tr}, and predictions $\hat{y}_{tr}^{(a)}$. Then

$$S_2 = \frac{1}{\sum_t m_t} \sum_t \sum_{r=1}^{m_t} \mathbf{1}\left\{ \hat{y}_{tr}^{(1)} = y_{tr} \text{ or } \hat{y}_{tr}^{(2)} = y_{tr} \right\}. \tag{1}$$

An all-outputs-correct task rate is a useful separate measure, but differs from Equation 1 when tasks have multiple test inputs. Pixel agreement is also a diagnostic, not a substitute: copying a large background can look close while missing the transformation entirely.

ARC-AGI-2 is a static reasoning task. The official 2026 track requires a Kaggle notebook with no internet during evaluation [5]. An externally hosted reasoning service therefore represents a different execution setting. Scores cannot be compared fairly without identifying the dataset, information access, attempt allowance, and resource limits.

3 How Published Solvers Approach the Problem

3.1 Induction and direct prediction

Induction searches for a transformation that can be executed on a new input. Transduction predicts the output directly from the demonstrations and test input. Li et al. study both using neural models trained on synthetic tasks generated from Python programs. They find complementary strengths: explicit programs handle precise computations and composition, while direct prediction can handle perceptual patterns that are harder to express in the searched program space [6]. This is evidence for combining mechanisms on their studied tasks, not a guarantee that any ensemble improves ARC-AGI-2.

The distinction is functional, not a division between neural and symbolic research. A neural model can propose executable programs. Conversely, a network trained on program-generated examples can predict grids without returning a program. For analysis, the important question is whether the candidate is an explicit mapping that can be checked on every demonstration.

3.2 Adaptation through weight updates

Akyürek et al. investigate test-time training: temporarily updating a model for the current task. Their procedure builds auxiliary examples from the supplied demonstrations, including leave-one-out constructions and transformations, and trains task-specific adapters. Initial fine-tuning, the adaptation data, and prediction aggregation all matter [7]. These experiments concern the earlier ARC benchmark and should not be presented as ARC-AGI-2 results. The transferable idea is that reasoning effort can be spent adapting parameters, not only sampling longer answers.

This distinction also clarifies leakage. Learning from the current task's permitted demonstrations is legitimate adaptation. Selecting hyperparameters after observing its hidden answers is evaluation feedback. Both change the effective system, but only the first is part of the task interface.

3.3 Refinement through recurrent computation

The Tiny Recursive Model repeatedly updates a latent state and an answer representation using a small shared network, with supervision across refinement steps [8]. Recurrence reuses parameters while increasing the computation applied to a problem. It therefore differs from both generating explicit programs and updating weights during inference. Parameter count alone cannot summarize its computational cost.

4 Why Checking Examples Is Necessary but Insufficient

Consider a task with demonstrations $D = \{(x_i, y_i)\}_{i=1}^{n}$ and a hypothesis class $\mathcal{H}$ of candidate programs. Its demonstration- consistent set is

$$\mathcal{V}(D) = \{h \in \mathcal{H} : h(x_i) = y_i \text{ for all } i\}. \tag{2}$$

This notation formalizes the check used in program-based reasoning; it is not a new solver. Membership proves agreement on the observed examples, not identification of the intended rule. Distinct programs can belong to $\mathcal{V}(D)$ and disagree on a test input. If the hypothesis class is unrestricted, even a lookup table with arbitrary behavior elsewhere fits every demonstration.

For a simple logical illustration, suppose each demonstrated object is both the largest object and blue. Selecting the largest object and selecting the blue object are then indistinguishable on those examples. An input where the properties separate distinguishes the rules. This is an invented illustration of underdetermination, not a reported ARC task or experiment. It explains why verification needs useful priors as well as execution.

Representations impose such priors before search begins. Treating a grid as connected objects, rows, or repeated motifs exposes different candidate rules. A representation that merges distinct objects can prevent the right rule from being expressed, even if the search procedure works perfectly. These are deductions about the pipeline, not measured claims that one representation is universally superior.

4.1 The second attempt is a selection problem

Let $c_{tr}^{(a)}$ indicate whether attempt a is correct and let S_1 use only the first attempt. With $N = \sum_t m_t$, the scoring rule gives the exact identity

$$S_2 - S_1 = \frac{1}{N} \sum_{t,r} (1 - c_{tr}^{(1)}) c_{tr}^{(2)}. \tag{3}$$

No independence assumption is needed. The second attempt helps only when it succeeds where the first fails. Two different explanations producing the same grid offer no extra exact-match coverage. Different grids offer a chance of improvement, not evidence that either is correct.

This gives a concrete reason to separate candidate quality from selection quality. A solver may generate the correct grid but discard it during ranking. An evaluator with reference answers can compute whether the candidate pool contained a correct answer, but that oracle diagnostic must not be reported as the submitted two-attempt score. The gap identifies a selection problem; its absence does not prove that generation is sound.

5 Reading Evidence Without Mixing Evaluation Settings

The ARC Prize 2025 technical report records the leading private-set scores in Table 2. These figures belong to the 2025 competition, not an experiment conducted here and not the outcome of the 2026 event. The report highlights test-time adaptation, synthetic training data, and refinement among successful approaches [9].

Table 2. Published ARC Prize 2025 private ARC-AGI-2 scores, transcribed without rounding from Table 1 of the organizers' report [9].

Entry	Private score (%)
NVARC	24.03
the ARChitects	16.53
MindsAI	12.64

These end-to-end results do not isolate architecture from pretraining, augmentation, or compute. Public ARC-AGI-1 scores likewise cannot establish progress on private ARC-AGI-2 tasks. Mechanistic insights can inform another setting; numerical comparisons must preserve the original conditions.

5.1 Accuracy and computation should travel together

For a solver M under a specified per-task compute cap B, let $S_2(M;B)$ be its score and $c_t(M;B)$ its observed cost on task t. A useful report retains both quantities:

$$(S_2(M;B),\ \bar{c}(M;B)), \qquad \bar{c}(M;B) = \frac{1}{T}\sum_{t=1}^{T} c_t(M;B). \tag{4}$$

Declare the cost unit and hardware; separate pretraining from task-time computation. The cap and observed average are not interchangeable. Equation 4 defines reporting quantities, not measured results.

5.2 Protecting the meaning of held-out evaluation

Public evaluation tasks should not guide development [3]. Record the source tasks for synthetic examples and keep their variants on the same side of the development/evaluation split.

Freeze both methods and evaluate the same tasks, retaining paired outcomes, seeds, timeouts, and malformed outputs. For uncertainty intervals, resample whole tasks with their test outputs and recompute the output-weighted score. This proposed protocol preserves within-task dependence; it cannot remove contamination or establish representativeness beyond the benchmark.

6 A Research Framework for Interpretable Progress

A follow-up study should compare checking and selection rules on a fixed candidate pool, then compare adaptation or recursion under a fixed resource cap. These proposed experiments separate changes in reasoning from simply running more search. No such comparison was conducted for this paper.

Table 3 makes failures actionable: did the system misread the input, fail to generate a useful rule, or select the wrong candidate?

Table 3. Proposed failure-analysis record. These are recommended diagnostic categories, not observations from an evaluated solver.

Stage	Evidence to retain	What it can distinguish
Representation	Parsed shapes and object relations	Input interpretation errors.
Generation	Candidates and their provenance	Search coverage versus format failure.
Checking	Per-demonstration execution results	Contradictions versus consistent hypotheses.
Selection	Ranked, distinct output grids	Missed correct candidates versus duplicates.
Execution	Timeouts, memory and runtime	Resource failures versus wrong predictions.

6.1 What this synthesis supports

The literature motivates several forms of task-specific computation. Our analysis distinguishes consistent candidates from identified rules, and useful candidate pools from effective selection. These deductions do not establish a performance gain for any new combination of methods.

This paper contains no implementation, dataset audit, or independent replication. Methods were selected for relevance rather than exhaustive coverage, and the historical table is not a current leaderboard. Exact grid prediction also samples a restricted problem class: success alone cannot establish general intelligence, real-world reliability, or equivalence to human reasoning. The cited versions identify evidence for future replication.

6.2 Conclusion

ARC-AGI-2 requires more than fitting visible examples. Program induction, neural adaptation, and recurrent refinement place computation at different stages and need comparison under matched conditions. Consistency checks, useful candidate diversity, and transparent evaluation explain more than an isolated score. A useful advance identifies which failure was removed, and the resources and prior information required to remove it.

References

1. Chollet, F.: On the measure of intelligence. arXiv:1911.01547v2 (2019). https://arxiv.org/abs/1911.01547v2
2. Chollet, F., Knoop, M., Kamradt, G., Landers, B., Pinkard, H.: ARC-AGI-2: A new challenge for frontier AI reasoning systems. arXiv:2505.11831v2 (2026; first released 2025). https://arxiv.org/abs/2505.11831v2
3. ARC Prize: ARC-AGI-2 official task repository and README. https://github.com/arcprize/ARC-AGI-2 (accessed 1 November 2026).
4. ARC Prize: ARC Prize 2026—ARC-AGI-2. Competition evaluation and submission format. https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2 (accessed 1 November 2026).
5. ARC Prize: ARC-AGI-2 competition, 2026 track description and scoring. https://arcprize.org/competitions/2026/arc-agi-2 (accessed 1 November 2026).
6. Li, W.-D., Hu, K., Larsen, C., et al.: Combining induction and transduction for abstract reasoning. arXiv:2411.02272v1 (2024). https://arxiv.org/abs/2411.02272v1
7. Akyürek, E., Damani, M., Qiu, L., Guo, H., Kim, Y., Andreas, J.: The surprising effectiveness of test-time training for abstract reasoning. arXiv:2411.07279v1 (2024). https://arxiv.org/abs/2411.07279v1
8. Jolicoeur-Martineau, A.: Less is more: Recursive reasoning with tiny networks. arXiv:2510.04871v1 (2025). https://arxiv.org/abs/2510.04871v1
9. Chollet, F., Knoop, M., Kamradt, G., Landers, B.: ARC Prize 2025: Technical report. arXiv:2601.10904v1 (2026). https://arxiv.org/abs/2601.10904v1